

Day 47 — Line-by-Line Syntax Explanation

day47_lenet5_style_cnn.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 6 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula, a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system.

MATH box legend: $\times$ = matrix multiply / elementwise times W, b = weights/bias m = batch size k = kernel size f_1, f_2 = filter counts $size$ = pooling window side length

Segment 0 — Imports and Global Setup

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)
```

Same imports every script in this series has used since Day 36. `matplotlib.use("Agg")` selects a non-interactive backend so the script can save PNG files to disk without needing a display -- required in this sandbox, harmless on a learner's own machine too. The global seed makes the module's own top-level randomness reproducible; every class below additionally takes its own `seed=42` parameter for its specific weight initialization.

Segment 1 — Shared Helpers (reused verbatim from Days 39-46)

```
def relu(z): return np.maximum(0, z)
def drelu(z): return (z > 0).astype(z.dtype)
def tanh(z): return np.tanh(z)
def dtanh(z): return 1 - np.tanh(z) ** 2
def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)
def one_hot(y, n_classes): ...
def cross_entropy_loss(probs, y_onehot): ...
def he_scale(n_in): return np.sqrt(2.0 / n_in)
def conv2d_forward_mc(X, W, b): ...
def conv2d_backward_mc(dZ, X, W): ...
def maxpool_forward(A, size=2, stride=2): ...
def maxpool_backward(dOut, A, idx_cache, size=2, stride=2): ...
def make_junction_dataset_rgb(...): ...
class TwoBlockConvNetRGB: ...
```

None of this is new today -- every function and class here is a character-for-character copy of Day 43-46's own code, unchanged. TwoBlockConvNetRGB is today's comparison baseline: ReLU convolutions, max pooling, one hidden dense layer. make_junction_dataset_rgb is copied verbatim rather than reimplemented from memory specifically because an RNG-order slip here has caused a real, hard-to-spot dataset mismatch before (Day 44) -- copying the exact function eliminates that risk entirely.

Segment 2 — Average Pooling (NEW today)

MATH

Forward, for a $\text{size} \times \text{size}$ window W :

$$\text{output} = (1 / \text{size}^2) \cdot \sum W_{i,j}$$

Backward, for output gradient g flowing into that window:

$\partial L / \partial W_{i,j} = g / \text{size}^2$ for EVERY (i,j) in the window (an equal share, unlike maxpool's all-or-nothing rule)

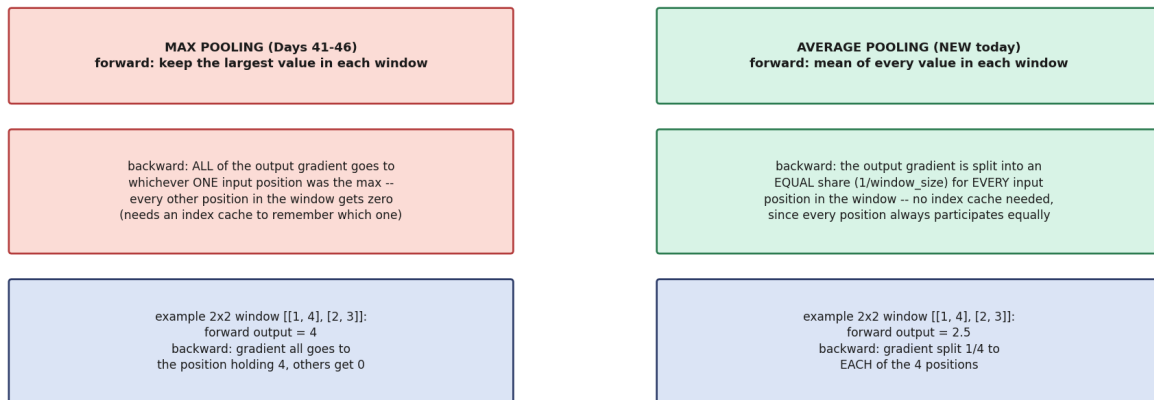
```
def avgpool_forward(A, size=2, stride=2):
    m, C, H, W = A.shape
    out_H = (H - size) // stride + 1
    out_W = (W - size) // stride + 1
    out = np.zeros((m, C, out_H, out_W))
    for i in range(out_H):
        for j in range(out_W):
            window = A[:, :, i*stride:i*stride+size,
                       j*stride:j*stride+size]
            out[:, :, i, j] = window.mean(axis=(2, 3))
    return out
```

Same sliding-window loop structure as `maxpool_forward`, but `window.mean(axis=(2, 3))` replaces `window.reshape(...).max(axis=2)` -- averaging over the last two axes (the window's own height and width) collapses each window down to one number per image, per channel. No index cache is created here, because average pooling's backward pass (below) doesn't need to know which position 'won' -- nothing wins; every position always contributes equally.

```
def avgpool_backward(dOut, A_shape, size=2, stride=2):
    m, C, H, W = A_shape
    dA = np.zeros(A_shape)
    out_H, out_W = dOut.shape[2], dOut.shape[3]
    share = 1.0 / (size * size)
    for i in range(out_H):
        for j in range(out_W):
            dA[:, :, i*stride:i*stride+size,
                j*stride:j*stride+size] += (
                dOut[:, :, i, j][:, :, None, None] * share)
    return dA
```

Notice this function's signature takes `A_shape` (a tuple), not the full input array `A` -- unlike `maxpool_backward`, average pooling's backward pass never needs to look at the original input VALUES at all, only its shape, because the gradient split is the same constant fraction (`share`) everywhere regardless of what the original values were. The two `[:, :, None, None]` reshape the per-window gradient scalar so it broadcasts correctly against the $\text{size} \times \text{size}$ block of `dA` it's being added into with `+=`.

SYSTEM DIAGRAM



SYSTEM DIAGRAM: the same 2x2 window through both pooling rules, forward and backward.

Segment 3 — LeNet5Style: __init__

```
class LeNet5Style:
    def __init__(self, img_size=30, n_classes=4, lr=0.08, seed=42):
        rng = np.random.RandomState(seed)
        self.lr = lr
        k = 5
        f1, f2 = 6, 16
        self.Wc1 = rng.randn(f1, 3, k, k) * he_scale(3 * k * k)
        self.bc1 = np.zeros(f1)
        self.Wc2 = rng.randn(f2, f1, k, k) * he_scale(f1 * k * k)
        self.bc2 = np.zeros(f2)
```

k=5, f1=6, f2=16 are LOCAL variables, not constructor parameters -- these are LeNet-5's own historical values, hardcoded on purpose because this class exists specifically to recreate that architecture, not to be a general-purpose configurable conv net the way TwoBlockConvNetRGB's f1/f2/k parameters are.

```
out1 = img_size - k + 1
pool1 = out1 // 2
out2 = pool1 - k + 1
pool2 = out2 // 2
self.flat_dim = f2 * pool2 * pool2
self.W1 = rng.randn(self.flat_dim, 120) * he_scale(self.flat_dim)
self.b1 = np.zeros(120)
self.W2 = rng.randn(120, 84) * he_scale(120)
self.b2 = np.zeros(84)
self.W3 = rng.randn(84, n_classes) * he_scale(84)
self.b3 = np.zeros(n_classes)
```

For img_size=30: out1=26, pool1=13, out2=9, pool2=4, so flat_dim=16*4*4=256. Three weight matrices now (W1, W2, W3) instead of the two every prior day's network has used -- this is what actually implements LeNet-5's 120 and 84-unit hidden layers. The same he_scale(n_in) initialization convention from every prior day carries over unchanged; only the LAYER SIZES being initialized are new.

Segment 4 — LeNet5Style: forward and backward

```
def forward(self, X):
    self.X = X
    self.Zc1 = conv2d_forward_mc(self.X, self.Wc1, self.bc1)
    self.Ac1 = tanh(self.Zc1)
    self.pooled1 = avgpool_forward(self.Ac1, 2, 2)
    self.Zc2 = conv2d_forward_mc(self.pooled1, self.Wc2, self.bc2)
    self.Ac2 = tanh(self.Zc2)
    self.pooled2 = avgpool_forward(self.Ac2, 2, 2)
```

The first structural difference from `TwoBlockConvNetRGB.forward()`: `tanh(self.Zc1)` where the baseline used `relu(self.Zc1)`, and `avgpool_forward` where the baseline used `maxpool_forward` (and correspondingly does NOT need to capture a `masks1` return value, since average pooling has no index cache to keep).

```
m = X.shape[0]
self.flat = self.pooled2.reshape(m, -1)
self.Z1 = self.flat @ self.W1 + self.b1
self.A1 = tanh(self.Z1)
self.Z2 = self.A1 @ self.W2 + self.b2
self.A2 = tanh(self.Z2)
self.Z3 = self.A2 @ self.W3 + self.b3
self.probs = softmax(self.Z3)
return self.probs
```

Three dense layers chained: flat -> Z1/A1 (120 units, tanh) -> Z2/A2 (84 units, tanh) -> Z3 (n_classes, no activation yet) -> softmax(Z3). `TwoBlockConvNetRGB`'s `forward()` stops after ONE hidden layer (flat -> Z1/A1 -> Z2 -> softmax) -- this extra A2 stage is the concrete code difference behind LeNet-5's '3 FC layers' design choice.

```
def backward(self, y_onehot):
    m = y_onehot.shape[0]
    dZ3 = self.probs - y_onehot
    dW3 = self.A2.T @ dZ3 / m
    db3 = dZ3.sum(axis=0) / m
    dA2 = dZ3 @ self.W3.T
    dZ2 = dA2 * dtanh(self.Z2)
    dW2 = self.A1.T @ dZ2 / m
    db2 = dZ2.sum(axis=0) / m
    dA1 = dZ2 @ self.W2.T
    dZ1 = dA1 * dtanh(self.Z1)
    dW1 = self.flat.T @ dZ1 / m
    db1 = dZ1.sum(axis=0) / m
```

Standard backprop chain rule, now three layers deep instead of two: softmax+cross-entropy's combined gradient (`probs - y_onehot`) starts the chain at `dZ3`, then each layer's `dtanh(...)` multiplies in that layer's own local derivative before the next `@ self.W_.T` propagates the error one layer further back. This is exactly the same pattern used since Day 37 -- just written out one more time for the extra layer.

```

dflat = dZ1 @ self.W1.T
dpooled2 = dflat.reshape(self.pooled2.shape)
dAc2 = avgpool_backward(dpooled2, self.Ac2.shape, 2, 2)
dZc2 = dAc2 * dtanh(self.Zc2)
dWc2, dbc2, dpooled1 = conv2d_backward_mc(
    dZc2, self.pooled1, self.Wc2)
dAc1 = avgpool_backward(dpooled1, self.Ac1.shape, 2, 2)
dZc1 = dAc1 * dtanh(self.Zc1)
dWc1, dbc1, _ = conv2d_backward_mc(dZc1, self.X, self.Wc1)

```

Here the two other real differences from TwoBlockConvNetRGB show up: avgpool_backward (not maxpool_backward -- and note it's called with self.Ac2.shape rather than needing self.Ac2 and a masks2 argument, since it doesn't need the actual values or a saved index cache) and dtanh(...) (not drelu(...)) applied at both conv layers, matching forward()'s own tanh() choice there.

```

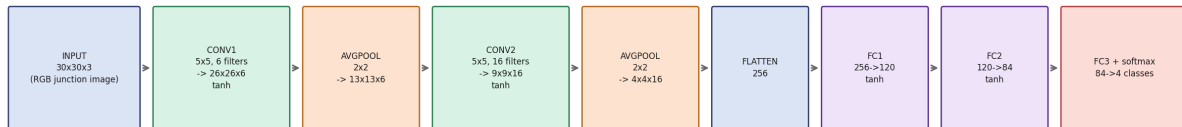
for p, g in [(self.Wc1, dWc1), (self.bc1, dbc1),
             (self.Wc2, dWc2), (self.bc2, dbc2),
             (self.W1, dW1), (self.b1, db1),
             (self.W2, dW2), (self.b2, db2),
             (self.W3, dW3), (self.b3, db3)]:
    p -= self.lr * g

```

The same plain-SGD update loop every network in this series has used since Day 37 -- now with 10 (parameter, gradient) pairs instead of 8, for the two extra tensors (W3, b3) LeNet-5's third FC layer adds.

SYSTEM DIAGRAM

LeNet-5 (LeCun et al., 1998), adapted from 32x32 grayscale / 10 classes to this series' 30x30 RGB / 4-class junction dataset



Faithful to the original: 5x5 kernels, 6 then 16 filters, tanh throughout, average pooling, three FC layers (120 -> 84 -> classes)

SYSTEM DIAGRAM: LeNet5Style's full forward shape trace, matching every line above.

Segment 5 — train(), accuracy(), param_count()

```
def train(self, X, y, epochs, n_classes=4):
    y_oh = one_hot(y, n_classes)
    losses = []
    for _ in range(epochs):
        probs = self.forward(X)
        losses.append(cross_entropy_loss(probs, y_oh))
        self.backward(y_oh)
    return losses
```

A convenience method wrapping the same full-batch forward/loss/backward loop every prior day has written out by hand at the call site. Calling `net.train(X, y, epochs=400)` directly, in one call, is exactly how a learner running this script on their own machine would use it -- no sandbox-specific chunking or checkpointing lives inside the shipped script itself.

```
def accuracy(self, X, y):
    probs = self.forward(X)
    preds = probs.argmax(axis=1)
    return (preds == y).mean()

def param_count(self):
    return sum(p.size for p in [self.Wc1, self.bc1, self.Wc2,
                                self.bc2, self.W1, self.b1, self.W2, self.b2,
                                self.W3, self.b3])
```

Both identical in spirit to `TwoBlockConvNetRGB`'s own versions, just extended: `param_count()` now sums over 10 tensors (including `W3/b3`) instead of 8. This is what produces today's headline 44,216 figure -- 21.8x the baseline's 2,024.

Segment 6 — Gradient Check (full pipeline, not an isolated function)

```
def demo_gradient_check():
    net = LeNet5Style(img_size=20, n_classes=3, lr=0.01, seed=1)
    X = rng.randn(4, 3, 20, 20) * 0.5
    y_oh = one_hot(np.array([0, 1, 2, 0]), 3)
    net.forward(X)
    ... # analytic gradients captured by hand-tracing backward()'s math
    ...
    eps = 1e-4
    for name, param in [("Wc1", net.Wc1), ("W1", net.W1), ("W3", net.W3)]:
        for _ in range(3):
            idx = tuple(rng.randint(0, s) for s in param.shape)
            orig = param[idx]
            param[idx] = orig + eps
            lp = cross_entropy_loss(net.forward(X), y_oh)
            param[idx] = orig - eps
            lm = cross_entropy_loss(net.forward(X), y_oh)
            param[idx] = orig
            numeric = (lp - lm) / (2 * eps)
```

img_size=20 (not the real 30) is deliberate -- small enough that the check runs fast, but still large enough that BOTH conv+avgpool blocks execute for real, exercising the actual new code today adds. For each of 3 random weight positions per tensor, the weight is nudged by +eps and -eps, the FULL forward() pipeline is rerun both times, and the resulting loss difference divided by 2*eps gives a numerical estimate of that single weight's gradient -- to compare against backward()'s own analytic answer for that same position.

```
ana = analytic[name][idx]
rel_err = abs(numeric - ana) / max(abs(numeric) + abs(ana), 1e-8)
max_rel_err = max(max_rel_err, rel_err)
```

The relative-error formula (not a plain difference) stays meaningful whether the true gradient is large or tiny -- dividing by the SUM of both magnitudes (with a 1e-8 floor to avoid dividing by exactly zero) normalizes the comparison. Today's actual worst case across all 9 checked positions came out to 4.22e-09, far below the roughly 1e-4 threshold that would flag a real problem.

Segment 7 — Training and Head-to-Head Comparison

```
def demo_train_and_compare():
    X_train, y_train = make_junction_dataset_rgb(
        n_per_class=50, size=30, arm=8, noise=0.4, channel_noise=0.15,
        center_jitter=2, seed=42)
    X_test, y_test = make_junction_dataset_rgb(
        n_per_class=20, size=30, arm=8, noise=0.4, channel_noise=0.15,
        center_jitter=2, seed=999)

    lenet = LeNet5Style(img_size=30, n_classes=4, lr=0.08, seed=42)
    losses_lenet = lenet.train(X_train, y_train, epochs=400)
    train_acc_lenet = lenet.accuracy(X_train, y_train)
    test_acc_lenet = lenet.accuracy(X_test, y_test)
```

The EXACT dataset convention Days 44 and 46 used: train is one generation (seed=42, 200 images across all 4 classes), test is a SEPARATE, independently-generated set (seed=999, 80 images) -- not a held-out split of the same generation. Getting this convention wrong (mixing it up with a different day's convention) is exactly the kind of mistake this series has flagged before, so it's copied here exactly rather than approximated.

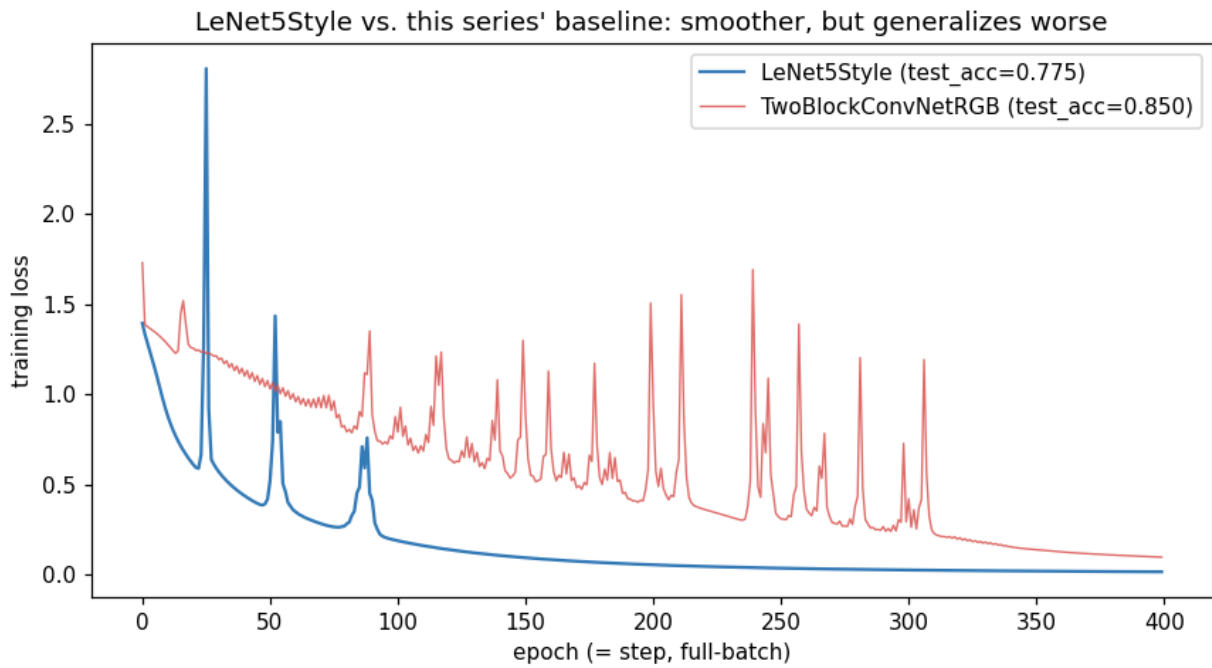
```
base = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                           lr=0.08, seed=42)
losses_base = base.train(X_train, y_train, epochs=400)
train_acc_base = base.accuracy(X_train, y_train)
test_acc_base = base.accuracy(X_test, y_test)
```

The comparison network -- same lr, same epoch count, same training data, same seed=42 initialization scheme -- differing ONLY in architecture (ReLU/max-pool/1-hidden-layer vs. LeNet5Style's tanh/avg-pool/3-hidden-layer design). Isolating architecture as the one changed variable is what makes the resulting comparison fair.

```
arr_lenet, arr_base = np.array(losses_lenet), np.array(losses_base)
noise_lenet_1st = float(np.std(np.diff(arr_lenet[:200])))
noise_lenet_2nd = float(np.std(np.diff(arr_lenet[200:])))
noise_base_1st = float(np.std(np.diff(arr_base[:200])))
noise_base_2nd = float(np.std(np.diff(arr_base[200:])))
```

`np.diff(arr)` computes consecutive differences (`loss[1]-loss[0]`, `loss[2]-loss[1]`, ...); the standard deviation of THAT array is a single number summarizing how much the loss jumps around step to step -- a smooth, steadily-decreasing curve gives a small value, a spiky curve gives a large one. Splitting into `[:200]` and `[200:]` (first half vs. second half of the 400 epochs) is what reveals each network's noise pattern CHANGES over the course of training, rather than reporting one flat number for the whole run.

SYSTEM DIAGRAM



The real, actually-rerun 400-epoch loss curves both networks produced.

Segment 8 — What This Maps To in PyTorch

```
def note_on_pytorch():
    print(
        "class LeNet5(nn.Module):\n"
        "    self.conv1 = nn.Conv2d(3, 6, kernel_size=5)\n"
        "    self.conv2 = nn.Conv2d(6, 16, kernel_size=5)\n"
        "    self.pool = nn.AvgPool2d(2, 2)\n"
        "    self.fc1 = nn.Linear(16 * 4 * 4, 120)\n"
        "    self.fc2 = nn.Linear(120, 84)\n"
        "    self.fc3 = nn.Linear(84, n_classes)"
    )
```

A pure documentation/teaching function -- it prints a PyTorch equivalent as a multi-line string rather than actually importing torch, exactly the same pattern every prior day's own `note_on_pytorch()`-style function has used. `nn.AvgPool2d` is the direct framework equivalent of today's `avgpool_forward/avgpool_backward`, with autograd computing the backward pass automatically instead of it being written out by hand.

```
def main():
    demo_gradient_check()
    demo_train_and_compare()
    note_on_pytorch()

if __name__ == "__main__":
    main()
```

The script's own entry point, calling all three demo functions in the exact order this document has walked them: verify correctness first, then produce the real trained results, then relate it all back to a production framework.

Quick Syntax Glossary — New or Reinforced Constructs

`window.mean(axis=(2, 3))` — Averages over TWO axes at once (a window's own height and width), collapsing a (m, C, size, size) slice down to (m, C) in one call -- new today, replacing `maxpool_forward's .reshape(...).max(axis=2)` pattern.

`dOut[:, :, i, j][:, :, None, None]` — Two chained indexing operations: first pulls out one (m, C) slice of the gradient array, then `[:, :, None, None]` inserts two new length-1 axes so the result broadcasts against a (m, C, size, size) block when multiplied by `avgpool_backward's` constant ``share``.

`A_shape (a tuple, not an array)` — `avgpool_backward's` second parameter is deliberately just a shape tuple, not the actual array -- a concrete signal that this function's math never needs to look at the original input's VALUES, only its dimensions, unlike `maxpool_backward` which needs the real array to look up its saved index cache.

`self.A2 (a THIRD stored activation)` — `TwoBlockConvNetRGB's` `forward()` only ever stores `self.A1` as its one hidden-layer activation. `LeNet5Style` stores `self.A1` AND `self.A2` -- both needed later in `backward()`, since the extra FC layer means one more activation has to survive from `forward()` to `backward()`.

`net.train(X, y, epochs=400)` — The single, clean way this lesson's classes are actually meant to be called -- new today as an added convenience method on both `LeNet5Style` and `TwoBlockConvNetRGB`, wrapping the same per-epoch loop every earlier day wrote out manually at each call site.

End of Day 47 syntax reference. For the concepts these lines implement, see the main Day 47 study guide PDF.